

Model Fitting II

Today's agenda:

- Issues with model fitting (7.4)

Last time

We discussed different approaches for model fitting

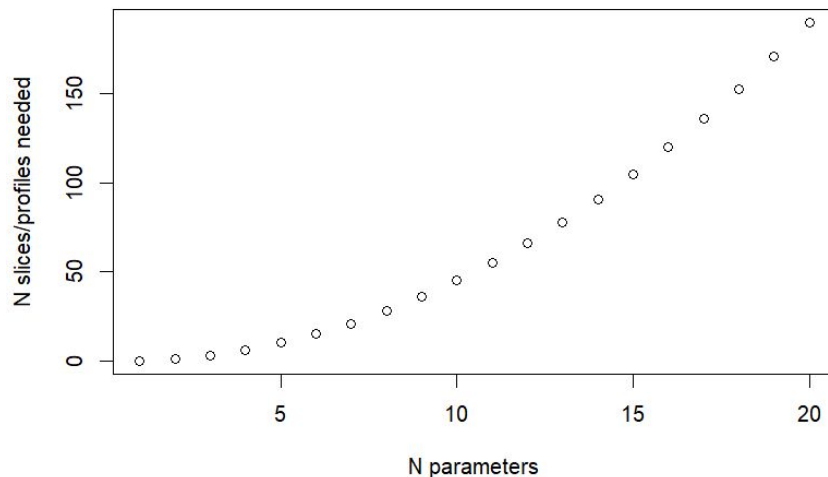
Today, we'll discuss some challenges you might encounter

High Dimensionality

- Touched on this last time
- With brute force, time to fit increases as a power law:
 - time per fit * number of values per parameter^{n parameters}
- Only possible to use brute force for a few parameters
- This is why we rely on faster algorithms
 - Even faster algorithms can be slow with lots of parameters

High Dimensionality

- Also causes issues for trying to understand likelihood surfaces
- 1 parameter model: likelihood function
- 2 parameter model: likelihood surface
- More than 2 parameter: need to use slices or profiles
 - This can be a lot of slices/profiles!



Slow function evaluations

- Slowness of high dimensional problems makes speed important!
- Most of the models we've fit take less than a second
- More complex models (or lots of models) can take MUCH longer!
- Not uncommon for advanced analyses to take weeks or months to run

Dealing with slow functions

- Try to use a faster algorithm
- Rewrite the code:
 - Apply is faster (but worse on memory)
 - for() is slower (but better on memory)
 - Use a simpler model
- Run analyses in parallel
- Switch to a better computer or cluster
- Wait a long time
 - make sure to include code to resume interrupted runs

Discontinuities and thresholds

Likelihood surfaces with many “flat” areas can cause problems

- E.g., if there is only a small area where the likelihood surface changes
- Derivative based metrics fail
- Other metrics will fail if not given the right starting parameters
 - because there is no likelihood variation to “climb”

Discontinuities and thresholds

Potential options:

- Try different algorithms, starting values
- Use profiles for parameters with these issues
- Modify the function to something smoother and easier to optimize

Multiple Minima

- Likelihood surfaces can have multiple minima
 - Optimizers can get stuck in one of these and give an erroneous answer
- You can check for this by using multiple algorithms, multiple starting values
 - If they don't agree, you might have multiple minima

Multiple Minima

Dealing with multiple minima:

- Check that models converged (this is reported by `optim` and `mle2`)
- Compare AICs of the different fits and take the best one
- Use likelihood profiles to examine the surface (including known minima)

Impossible Parameter values

Sometimes, there are constraints on parameter values

- E.g., total prey consumption has to be equal to consumption of each species

In some cases, model optimization will select values that don't make sense

- (This is why you should check your model predictions)

Impossible Parameter values

Dealing with impossible parameter values:

- Try different algorithms or starting values and hope you get something good
- Transform parameter values to avoid impossible values
 - E.g., log (keep parameters positive) and logit (keep between 0 and 1) transformations
- Use a method that allows you to constrain values (L-BFGS-B, nlminb)
- Modify your NLL function to penalize nonsensical values

Note: this is roughly the order I would recommend trying these in.

Example: returning to the tadpole from last time

```
library(emdbook)
library(bbmle)

data("ReedfrogFuncresp")

m2.bad_start <- mle2(
  Killed ~ dbinom(size = Initial,
                  prob = a / (1 + a * h * Initial)),
  start = list(a = 0.01, h = 10),
  data = ReedfrogFuncresp
)
```

Example: returning to the tadpole from last time

We can compare our model estimates using the “predict” function

```
plot(x = ReedfrogFuncresp$Initial,  
     y = ReedfrogFuncresp$Killed)
```

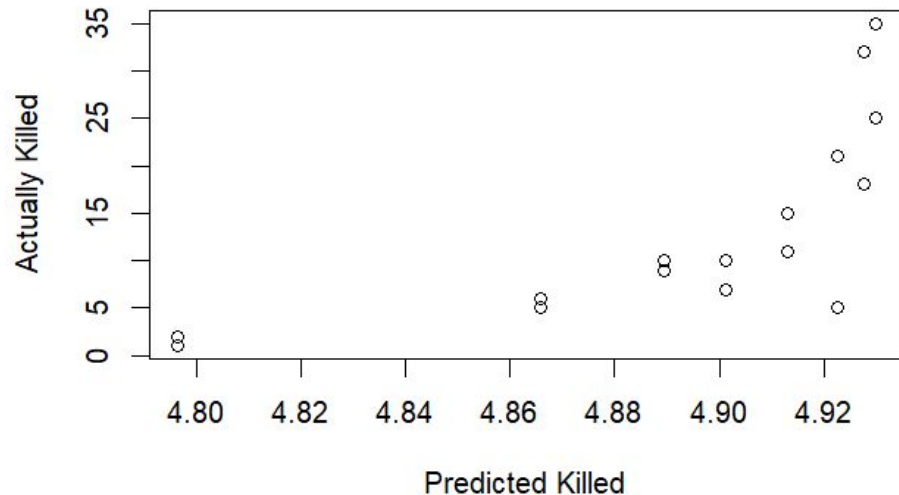
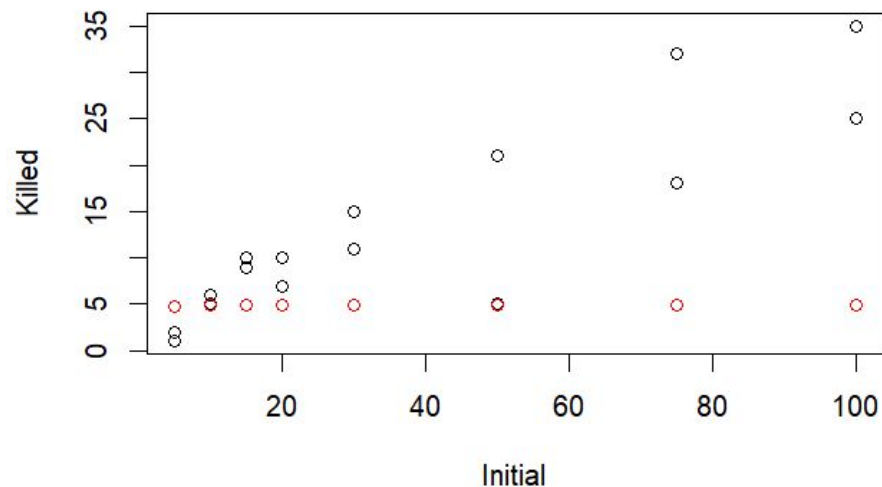
```
points(x = ReedfrogFuncresp$Initial,  
       y = predict(m2.bad_start),  
       col = "red")
```

Example: returning to the tadpole from last time

We might also plot the actual data against the predicted data

```
plot(x = predict(m2.bad_start),  
     y = ReedfrogFuncresp$Killed,  
     xlab = "Predicted Killed",  
     ylab = "Actually Killed")
```

Our model doesn't fit great



Profiling our model

Take a look at the parameters of your model: which value seems more extreme?

- It makes sense to start with whichever parameters seems less sensible

Profiling our model

Since a is around 33, we'll start with that one

First, make a vector of “ a ” values to check, then a vector to store output data:

```
a_profile_seq <- seq(from = 0.1,to = 40,by=0.1)
```

```
a_profile_nll <- numeric(length(a_profile_seq))
```

Profiling our model

Now, we calculate NLL for each a value in the vector and record the output

```
for(i in 1:length(a_profile_seq)){  
  
  out_i <- mle2(  
    Killed ~ dbinom(size = Initial,  
                    prob = a / (1 + a * h * Initial)),  
    start = list(a = 0.01, h = 10),  
    data = ReedfrogFuncresp,  
    fixed = c(a = a_profile_seq[i])  
  )  
  
  a_profile_nll[i] <- out_i@min  
  
} #end for loop
```

Run this loop. What happens?

Handling errors

- The optimization threw errors for some values, stopping the loop
- R has different error handling functions for these instances
- `tryCatch()` is among the most powerful
 - It allows you to “catch” the error

tryCatch

The usual way to use tryCatch is:

```
output <- tryCatch(expr = <some code to run>,  
                  error = function(e){e})
```

If there is an error, your output will be of class “error”

- We can then replace the output with NA or whatever we like

Revising our profile with tryCatch

```
for(i in 1:length(a_profile_seq)){  
  out_i<- tryCatch(  
  
    mle2( Killed ~ dbinom(size = Initial,  
                          prob = a / (1 + a * h * Initial)),  
      start = list(a = 0.01, h = 10),  
      data = ReedfrogFuncresp,  
      fixed = c(a = a_profile_seq[i])  
    ), error = function(e){e})  
  
  if(inherits(x = out_i, what = "error")){  
    a_profile_nll[i] <- NA  
  }else{  
    a_profile_nll[i] <- out_i@min  
  }  
}
```

Plotting our likelihood profile

```
plot(x = a_profile_seq,  
     y = a_profile_nll,  
     xlab = "a",  
     ylab = "NLL")
```

```
abline(v = m2.bad_start@coef["a"]) # add a line for our model fit estimate
```

Does our estimate for a correspond to the best NLL? What went wrong?

Plotting our likelihood profile

```
plot(x = a_profile_seq,  
     y = a_profile_nll,  
     xlab = "a",  
     ylab = "NLL")
```

```
abline(v = m2.bad_start@coef["a"]) # add a line for our model fit estimate
```

Does our estimate for a correspond to the best NLL? What went wrong?

Updating out model

We can get a new starting estimate for “a” based on our profile

```
best_guess_a <- a_profile_seq[which.min(a_profile_nll)]
```

Then we re-run the model with the new starting estimate

```
m2.good_start <- mle2(  
  Killed ~ dbinom(size = Initial,  
                  prob = a / (1 + a * h * Initial)),  
  start = list(a = best_guess_a, h = 10),  
  data = ReedfrogFuncresp  
)
```

Comparing the updated model

Check out the parameters

```
m2.bad_start
```

```
m2.good_start
```

Compare using aic

```
AICtab(m2.bad_start,m2.good_start)
```

Which model is best?

Plotting out new model results

Plot predictions on top of actual

```
plot(x = ReedfrogFuncresp$Initial,  
     y = ReedfrogFuncresp$Killed,  
     ylab = "Killed",  
     xlab = "Initial")
```

```
points(x = ReedfrogFuncresp$Initial,  
       y = predict(m2.good_start),  
       col = "red")
```

Plot killed vs predicted killed

```
plot(x = predict(m2.good_start),  
     y = ReedfrogFuncresp$Killed,  
     xlab = "Predicted Killed",  
     ylab = "Actually Killed")
```

Does the model predictions seem better?

Take-aways

Compare your model predictions to reality

If the model fit seems poor, try:

- Different optimizers or starting values
- profiles/slices
- Using constraints
- Transforming model parameters
- Using a different model

Before next class:

- Read 7.5